

LoRa Ping-Pong Link Test

How the code works - two identical XIAO ESP32-C3 + E32-433T30S units

Derived from PAMIANG_High_Power_WIFIAP_30s | Arduino-ESP32 core 2.0.x

1 Purpose & topology

This is a link-quality test for a pair of E32-433T30S LoRa radios. Two identical boards continuously bounce small packets off each other and report packet counts, loss, and round-trip time (RTT) on a built-in web page. It answers "is the radio link working, and how good is it?" without any PC tools - just a phone browser.

```
Unit 1 (ID=1)                                Unit 2 (ID=2)
+-----+                                +-----+
| XIAO C3 | PING ----->                | XIAO C3 |
| + E32-T30S | <----- PONG | + E32-T30S |
+-----+                                +-----+
WiFi AP: PingPong-Unit1                     WiFi AP: PingPong-Unit2
http://192.168.4.1                         http://192.168.4.1
```

Both units run the **same firmware**. The build flag PINGPONG_ID (1 or 2) is the only difference; from it the code derives MY_ID and PEER_ID. Each unit both pings and answers, so the test is fully symmetric.

2 Hardware & the E32 in transparent mode

The E32-433T30S is used in **transparent UART mode** (mode pins M0=M1=0). In this mode the module is a wireless serial cable: bytes written to the UART on one side come out of the UART on the other. There are no M0/M1/AUX control pins wired (same as the PAMIANG board), so the firmware just opens a 9600 8N1 serial port and reads/writes bytes. Both modules must share the same address & RF channel.

E32-433T30S	XIAO ESP32-C3	Note
TXD	D5 (Serial1 RX)	module -> MCU
RXD	D4 (Serial1 TX)	MCU -> module
VCC	3V3	TX bursts pull ~0.6-0.8 A; use a stout supply
GND	GND	
M0, M1	GND	transparent / normal mode

3 Packet format

Because a UART is a continuous byte stream, packets are framed by a magic start byte and a fixed length, and validated by an XOR checksum. Both ends are the same little-endian MCU, so the C struct is sent as raw bytes (19 bytes total).

Field	Type	Meaning
magic	uint8	0x50 ('P') - frame start / sync
type	uint8	1 = PING, 2 = PONG
src	uint8	sender unit id
dst	uint8	target unit id
seq	uint16	sequence number (per-sender, increments)
t_ms	uint32	pinger's send-time (millis); echoed in PONG
src_sent	uint32	sender's total pings sent (peer monitoring)
src_rt	uint32	sender's total round trips (peer monitoring)
crc	uint8	XOR of all preceding bytes

src_sent / src_rt let a receiver display the *other* unit's counters too, so either portal shows both stations' status from a single packet.

4 Message flow & round-trip timing

```
Unit 1 (pinger)                                Unit 2 (responder)
t_ms = millis()  --- PING(seq,t_ms) -->;
awaiting = true                                     receives PING (dst==2)
                                                    pingsRecv++
                                                    &lt;--- PONG(seq,t_ms) --- replies, ECHOING t_ms unchanged
receives PONG (seq matches awaitSeq)
RTT = millis() - t_ms  &lt;--- t_ms is U1's own stamp, so no clock sync needed
roundTrips++ ; awaiting = false
```

The key trick: the responder echoes the pinger's **t_ms** verbatim. When the PONG returns, the pinger subtracts that stamp from its current millis() to get the full round-trip time. Because the same clock stamps and measures, the two boards' clocks never need to be synchronized.

Loss detection

The pinger keeps one ping 'in flight' (awaiting + awaitSeq). Each PONG that matches awaitSeq clears it and records a round trip. If the next 2 s tick arrives while a ping is still awaiting, that ping is counted as **lost** and a new one is sent. Success rate = roundTrips / (roundTrips + lost).

```
every PING_INTERVAL_MS (2000 ms):
  if (awaiting) lost++           // previous ping never answered
  seq++ ; awaitSeq = seq ; awaiting = true ; pingsSent++
  sendPacket(PING, seq, millis())
```

5 Receive framing (pollLoRa)

Called every loop iteration. It reads available UART bytes and reassembles frames:

```
idx==0: discard bytes until one equals magic 0x50, then store it
idx>0 : append bytes until idx == 19 (full packet)
verify crc == XOR(first 18 bytes)
  ok  ->; handlePacket()  (PING ->; reply PONG ; PONG ->; compute RTT)
  fail ->; drop, resync on next magic byte
```

Only packets whose src == PEER_ID are accepted, so stray bytes or a third radio on the channel are ignored.

6 Web portal

In setup() each unit starts a WiFi SoftAP (PingPong-Unit<id>, password pingpong123) and a WebServer on port 80. Two routes:

Route	Returns
/	Self-contained HTML dashboard (polls /api every 1 s via fetch)
/api	JSON: ids, link_up, pings_sent, round_trips, lost, pings_recv, success_pct, last/min/max/avg RTT, peer_sent, peer_rt, ...

The page shows two cards - **This unit** (local counters, success rate, RTT) and **Peer** (the other unit's counters carried in packets) - plus a LINK UP/DOWN badge. Link is 'up' while a packet from the peer arrived within LINK_TIMEOUT_MS (6 s). No internet or app needed; just join the AP and browse.

7 Build, flash, monitor

```
pio run -e unitA -t upload    # board 1  (-DPINGPONG_ID=1)
pio run -e unitB -t upload    # board 2  (-DPINGPONG_ID=2)
# prebuilt: bin/PingPong_UnitA_id1...bin , bin/PingPong_UnitB_id2...bin
```

Power both boards. Within a couple seconds the serial log shows PING/PONG and the portals show LINK UP with a climbing success rate. Move the units apart to watch RTT rise and loss appear - that is your range/quality reading.

Tuning knobs (top of main.cpp): PING_INTERVAL_MS (ping cadence), LINK_TIMEOUT_MS (peer-down threshold), LORA_BAUD, and the D5/D4 LoRa pins.